

eBPF et io_uring : audit avancé — A-EPI-003

Vérifier l'oracle, suivre le proxy

Ted "Tedsig42" Kouhouenou ted.sig42@tutamail.com

Summary: Module avancé : analyser les passes du vérificateur eBPF, comprendre les écarts entre vérification et JIT, puis suivre le cycle de vie des ressources io_uring. Produire des règles, confirmer une course sans sleep() et analyser un patch gap.

Prerequisites : A-EPI-001, C noyau, assembleur x86-64, concurrence

Version: 1.0

Contents

I Instructions **2**

II Instructions Communes **5**

II.1 Structure du Dépôt 5

II.2 Qu'est-ce qu'une Preuve de Concept Valide 5

II.3 Processus de Soumission 6

II.4 Règle des Fichiers Supplémentaires Interdits 7

II.5 Format du Rapport 7

II.6 Cibles Autorisées Uniquement 8

II.7 Citez Toutes Vos Sources 8

III Instructions sur l'IA **10**

III.1 Aperçu de la Politique 10

III.2 Utilisations Autorisées 10

III.3 Utilisations Interdites 11

III.4 Responsabilité Personnelle 11

III.5 Citation Obligatoire de l'IA 11

III.6 Évaluation par les Pairs et l'IA 12

III.7 Rappel Éthique 13

IV Avant-propos **14**

V Aperçu du module **15**

VI Exercise 00 **16**

VII Exercise 01 **17**

VIII Exercise 02 **18**

IX Exercise 03 **19**

X Exercise 04 **21**

XI Rendu et évaluation **22**

XII Ressources par exercice **23**

XIII Sources **25**

Chapter I

Instructions

- Trois jours.
- Analyse statique et lectures de sources uniquement.
- Ne chargez pas de programme eBPF sur une machine partagée.
- Pour io_uring, utilisez des traces ou des tests unitaires, pas de reproduction d'exploit.

Format du rendu — en-tête obligatoire

Tout fichier rendu dans ce module doit commencer par l'en-tête ci-dessous, **adapté à la syntaxe de commentaire du langage** utilisé.

Modèle générique

```
/**
 * Module: <code module>
 * Exercise: <numéro exo>
 * File: <nom du fichier>
 * Author: <votre login JeuneCrack>
 *
 * Description: <une phrase, pas de roman>
 *
 * Return: <voir ci-dessous>
 */
```

Login

Votre identifiant **JeuneCrack** (ex: `tedsig42`). Pour un rendu en binôme : `login1/login2`. Ajoutez un handle Matrix après le login si vous le souhaitez (ex: `tedsig42 @tedsig42`).

Return — selon le type de fichier

Type de fichier	Valeur de Return
C (`.c`)	`return (0);` (succès), `return (-1);` (erreur), `return (valeur);` (résultat calculé)
Python (`.py`)	`0` (succès), `1` (erreur), `N/A` (analyse, script sans exit)
Shell (`.sh`)	`exit 0` (succès), `exit 1` (erreur)
Markdown (`.md`)	`N/A` — les fichiers d'analyse n'ont pas de valeur de retour
Exploit C/Python	`shell` (shell obtenu), `flag: <flag>` (flag capturé), `proof: <hash>` (preuve de concept)
Binaire (`.bin`)	`N/A` — pas d'en-tête pour les fichiers binaires

Exemples par langage

C (`.c`) :

```c
/**
 * Module: F-EXP-001
 * Exercise: ex02
 * File: exploit.c
 * Author: tedsig42
 *
 * Description: Buffer overflow with ret2libc
 *
 * Return: 0
 */
```

Python (`.py`) :

```python
# Module: F-WEB-001
# Exercise: ex05
# File: ssrf-exploit.py
# Author: tedsig42
#
# Description: Blind SSRF bypass via redirect
#
# Return: 0
```

Shell (`.sh`) :

```bash
# Module: F-SUP-001
# Exercise: ex03
# File: dependency-confusion.sh
# Author: tedsig42
#
# Description: Automated dependency confusion test
#
# Return: exit 0
```

Markdown (`.md`) — analyse, rapport :

```
<!--
Module: M-SRC-001
Exercise: ex04
File: audit-report.md
Author: tedsig42

Description: Code audit of vulnerable-httpd

Return: N/A
-->
```

Chapter II

Instructions Communes

II.1 Structure du Dépôt

Votre dépôt Git doit respecter la convention de nommage suivante pour tous les modules soumis. L'outil d'évaluation (`pendragon`) se base sur cette structure pour localiser et tester votre travail.

```
<login>/
+-- <MODULE-ID>-<slug>/         # ex. M-BLS-001-business-logic/
|   +-- <turnindir>/            # ex. ex00-pricing-manipulation/
|   |   +-- <file1>             # ex. poc-pricing.py
|   |   +-- <file2>             # ex. report.md
|   |   +-- ...
|   +-- <turnindir>/            # ex. ex01-race-conditions/
|   |   +-- ...
|   +-- ...
+-- ...
```

- **Répertoire du module** : le nom complet du répertoire tel qu'affiché dans votre curriculum (ex. `M-BLS-001-business-logic`).
- **Répertoire de rendu** : la valeur de `\turnindir` fournie dans chaque exercice (ex. `ex00-pricing-manipulation`).
- **Fichiers de l'exercice** : exactement les fichiers listés dans `\exfiles`. Ni fichiers supplémentaires, ni fichiers manquants.

II.2 Qu'est-ce qu'une Preuve de Concept Valide

Une PoC doit être :

1. **Reproductible**. N'importe qui avec le même environnement peut l'exécuter et observer la vulnérabilité. Les chemins codés en dur, les adresses IP absolues et les jetons expirés ne sont pas acceptables.

2. **Documentée**. Chaque étape est expliquée. L'évaluteur ne devrait pas avoir à deviner ce que fait le script.

3. **Ciblée**. L'exercice spécifie une cible autorisée (lab, maquette, binaire local ou plateforme désignée). Ne testez jamais les PoC sur des systèmes de production qui ne vous appartiennent pas.

4. **Minimale**. La PoC démontre *uniquement* la vulnérabilité en question. Les charges utiles non liées et le bruit sont pénalisés.

① Une PoC qui modifie silencieusement l'état du système, supprime des fichiers ou persiste des changements en dehors du périmètre de l'exercice sera rejetée. Si votre exploit nécessite un nettoyage, documentez les étapes de nettoyage dans votre rapport.

II.3 Processus de Soumission

1. Réalisez les livrables de l'exercice sur votre machine locale.

2. Committez et poussez vers votre dépôt Git assigné.

3. Le dépôt est gelé après la date limite.

4. Les pairs clonent votre dépôt et évaluent en utilisant :

   - `pendragon` pour l'exécution automatisée des tests (là où le module fournit des configurations mouli).
   - La grille manuelle pour la qualité du rapport, le style de code et la méthodologie.



Exécutez toujours `pendragon run <MODULE-ID> <ex_id> <path/to/student/repo>` avant votre commit final pour vérifier que votre soumission passe les vérifications automatisées.

Test avec pendragon

Si le module inclut une configuration mouli (fichiers JSON dans `resources/mouli/`), vous pouvez tester votre soumission localement :

```
pendragon run M-BLS-001 ex00 ~/student-repos/tedsig42
```

`pendragon` résout le chemin de l'exercice automatiquement en cherchant `<mod-fullname>/<ex-dirname>/` dans le répertoire étudiant donné.

Vérification de style avec caliban

Pour les modules qui nécessitent du code C, `caliban` vérifie la conformité du style :

```
caliban check ~/student-repos/tedsig42/M-BLS-001-business-logic
```

`caliban` applique la Norme 42 (taille des fonctions, nommage des variables, constructions interdites) adaptée aux contextes de recherche de vulnérabilités. Exécutez-le avant la soumission — un échec aux vérifications de style peut invalider l'exercice.

II.4 Règle des Fichiers Supplémentaires Interdits

- Votre dépôt doit contenir **uniquement** les fichiers listés dans `\exfiles` pour chaque exercice.
- Fichiers interdits : `.DS_Store`, `__pycache__`, `.venv/`, `node_modules/`, `.git/`, binaires compilés (sauf explicitement listés dans `exfiles`), fichiers d'échange d'éditeur, fichiers journaux et tout fichier non demandé.
- Utilisez `.gitignore` pour exclure automatiquement les fichiers indésirables.

II.5 Format du Rapport

Lorsqu'un rapport (`report.md`) est requis, il doit inclure :

1. **Classe de vulnérabilité** (identifiant CWE si applicable).
2. **Étapes de reproduction** — numérotées, prêtes à être copiées-collées.
3. **Charges utiles** — chaînes exactes, requêtes ou entrées utilisées.
4. **Impact** — ce qu'un attaquant peut accomplir.
5. **Mitigation** — comment le développeur devrait corriger.
6. **Sources** — toutes les ressources consultées (voir Section II.7).

En-tête de Rendu

Chaque fichier source doit inclure l'en-tête de rendu défini dans `../../../resources/latex/deliverable-header.tex`, adapté à la syntaxe de commentaire du langage utilisé.

II.6 Cibles Autorisées Uniquement

① Vous pouvez **uniquement** tester les cibles explicitement listées dans l'exercice (labs, serveurs locaux, maquettes, binaires désignés). Tester des systèmes de production réels, des applications tierces ou toute cible qui ne vous appartient pas ou pour laquelle vous n'avez pas d'autorisation écrite est interdit et peut entraîner une disqualification immédiate.

L'éthique du bug bounty s'applique tout au long de ce curriculum :

- Aucun test sur des programmes en production sans un accord de périmètre valide.
- Aucun déni de service sauf si l'exercice l'exige explicitement.
- Aucune exfiltration de données au-delà de ce qui est nécessaire pour démontrer la vulnérabilité.
- Si vous accédez accidentellement à des données réelles d'utilisateurs, arrêtez immédiatement et documentez ce qui s'est passé.

II.7 Citez Toutes Vos Sources

Toute ressource consultée pendant votre travail doit être citée. Cela inclut :

- Articles académiques et billets de blog.
- Rapports publics de bug bounty (HackerOne, Bugcrowd, etc.).
- Documentation officielle (RFC, pages de manuel, documentation fournisseur).
- **Outils d'IA** : nom du modèle, résumé de la question, date et méthode de vérification (voir le chapitre Instructions IA pour le format de citation exact).
- Extraits de code adaptés de projets open-source (avec attribution de licence si requis).

Format de citation

```
# R\'ef\'erences

- [1] PortSwigger, "Business Logic Flaws",
  https://portswigger.net/web-security/logic-flaws
  (consult\'e le 2025-03-15).
- [2] Rapport HackerOne #123456, "IDOR on /api/users",
  anonymis\'e, consult\'e via divulgation publique.
- [3] G\'en\'er\'e avec l'assistance de Claude 3.5 Sonnet
  le 2025-03-14. Question : "Write a Python script that...".
  V\'erifi\'e par : revue manuelle de chaque requ\^ete HTTP.
```

Attribution en ligne

Si un exercice vous demande explicitement d'utiliser une technique ou un outil spécifique, vous n'avez pas besoin de citer l'exercice lui-même comme source. Citez tout ce qui se trouve en dehors de l'énoncé de l'exercice.

Chapter III

Instructions sur l'IA

III.1 Aperçu de la Politique

Les outils de codage assistés par IA (ChatGPT, Claude, Copilot, Gemini, Cursor, Codeium, et similaires) sont **autorisés** tout au long de ce cursus. La recherche en sécurité est un domaine où la maîtrise de l'IA devient essentielle — apprendre à utiliser ces outils efficacement fait partie de votre formation.

Cependant, l'objectif est la **compréhension**, pas l'automatisation. Si vous ne pouvez pas expliquer chaque ligne de votre exploit sans l'aide de l'IA, vous n'avez pas terminé l'exercice.

① Utiliser l'IA pour contourner l'objectif pédagogique d'un exercice — par exemple, lui demander de rétro-ingéniérer un binaire pour ne pas avoir à apprendre le désassembleur, ou de générer une chaîne d'exploitation complète que vous ne comprenez pas — va à l'encontre du but du cursus et sera traité comme une fraude académique.

III.2 Utilisations Autorisées

- **Explication de concepts**. Demander à l'IA de clarifier une classe de vulnérabilité, un sous-système du noyau, ou un protocole réseau.
- **Échafaudage de code**. Générer du code de départ pour un harnais, un fuzzer, ou un squelette de PoC — puis le modifier et l'étendre vous-même.
- **Débogage**. Expliquer des messages d'erreur, la sortie d'un désassembleur, ou un comportement de crash inattendu.
- **Résumé de documentation**. Condenser une longue RFC ou un article technique en notes exploitables.
- **Traduction**. Traduire du contenu technique entre l'anglais et le français (les deux langues sont utilisées dans ce cursus).

- **Rédaction de rapports**. Demander de l'aide pour structurer un rapport de bug bounty — mais remplir le contenu technique vous-même.

III.3 Utilisations Interdites

- **Génération complète d'exploit**. Demander à l'IA d'écrire l'exploit complet d'un exercice, puis le soumettre sans le comprendre.
- **Contournement des contraintes de l'exercice**. Utiliser l'IA pour contourner une restriction que l'exercice impose explicitement (p. ex. « n'utilisez pas de scanners automatisés »).
- **Découverte automatisée de vulnérabilités**. Exécuter des outils de scan pilotés par IA qui remplacent l'analyse manuelle que vous êtes censé pratiquer.
- **Rédaction des grilles d'évaluation**. Vous n'êtes pas autorisé à utiliser l'IA pour générer des critères d'évaluation par les pairs ou des réponses.
- **Plagiat sans attribution**. Copier le résultat de l'IA sans le citer (voir Section III.5).

III.4 Responsabilité Personnelle

Lorsque vous demandez à l'IA de générer du code d'exploit, vous devenez responsable de chaque octet de ce résultat avant de l'utiliser contre une cible.

1. **Lisez le code généré** ligne par ligne.
2. **Identifiez toutes les hypothèses** faites par l'IA (architecture, version du noyau, drapeaux du compilateur, topologie réseau).
3. **Testez en isolation** avant d'exécuter contre une cible d'exercice.
4. **Comprenez les modes d'échec**. Si l'exploit généré plante ou se bloque, vous devez diagnostiquer et corriger le problème vous-même.



Si l'IA génère du code que vous ne comprenez pas, demandez-lui de l'expliquer. Si vous ne comprenez toujours pas après l'explication, le code n'est pas prêt à être utilisé. Passez à une approche plus simple que vous maîtrisez entièrement.

III.5 Citation Obligatoire de l'IA

Chaque fichier contenant du code généré ou assisté par l'IA doit inclure un commentaire de citation. Le format est :

```
# Generated with assistance from <tool> on <date>.
# Prompt: <one-line summary of the prompt>
# Verified by: <method used to verify the output>
```

Exemples

Python :

```
# Generated with assistance from Claude 3.5 Sonnet on 2025-03-14.
# Prompt: "Write a Python script that sends HTTP requests with
# modified price fields to /api/checkout/pay"
# Verified by: manual review of each request and response
```

C :

```
/* Generated with assistance from GitHub Copilot on 2025-03-14.
 * Prompt: "Implement a stack-based buffer overflow with
 * ret2libc for x86_64 Linux"
 * Verified by: compiled with -fno-stack-protector and
 * tested against local binary with ASLR disabled */
```

Markdown (rapport) :

```
<!--
Generated with assistance from ChatGPT-4 on 2025-03-14.
Prompt: "Structure a bug bounty report for IDOR with
sections: summary, steps, impact, mitigation"
Verified by: all technical content written manually,
AI only suggested structure
-->
```

Interactions multiples avec l'IA

Si vous avez utilisé plusieurs outils d'IA, ou itéré à travers plusieurs invites, citez chaque interaction significative. Un groupe d'invites connexes peut être résumé :

```
<!--
AI assistance used in this file:
- Claude 3.5 Sonnet (2025-03-14): harness skeleton generation
- ChatGPT-4 (2025-03-14): crash triage help
- Copilot (2025-03-15): inline autocomplete for loop logic
All generated code reviewed and modified manually.
-->
```

III.6 Évaluation par les Pairs et l'IA

Lors de l'évaluation par les pairs, l'évaluateur peut :

- Vous demander d'**expliquer tout code généré par l'IA** ligne par ligne.

- Vous demander de **reproduire le raisonnement** sans l'aide de l'IA.
- Demander à voir votre **historique d'invites** pour un exercice donné.
- Déduire des points si la citation de l'IA est manquante ou si vous ne pouvez pas expliquer le code que l'IA a généré.

① Présenter le code généré par l'IA comme votre propre travail (défaut de citation) est du plagiat. Si l'évaluateur soupçonne du plagiat, il peut saisir l'équipe pédagogique pour examen.

III.7 Rappel Éthique

- Les modèles d'IA peuvent produire du code incorrect, dangereux, ou illégal. Vous êtes **responsable** de vérifier chaque résultat avant exécution.
- Les modèles d'IA entraînés sur la recherche publique en sécurité peuvent reproduire textuellement du code provenant de projets sous licence GPL. Vérifiez les licences avant d'intégrer ces éléments dans votre travail.
- Ne saisissez pas de code sensible ou propriétaire dans des outils d'IA hébergés dans le cloud. Utilisez des modèles locaux (Llama, Mistral, DeepSeek) pour les travaux impliquant des cibles non publiques.
- L'éthique de la recherche en sécurité que vous apprenez ici s'applique à l'utilisation de l'IA : ne demandez pas à l'IA de générer des malwares, des rançongiciels, ou du code destiné à nuire à des systèmes en dehors du cadre de l'exercice.



La maîtrise des outils d'IA est une compétence recherchée dans la chasse aux bogues moderne. Utilisez-les sagement, citez-les honnêtement, et conservez toujours votre propre compréhension de chaque exploit que vous déployez.

Chapter IV

Avant-propos

Un vérificateur eBPF est un oracle : il promet qu'un programme respecte certaines propriétés avant de l'exécuter. Un oracle peut être :

- **localement correct** : chaque passe fait ce qu'elle dit ;
- **globalement faux** : la composition des passes ne préserve pas la propriété.

io_uring est un proxy concurrent : il ne promet pas que la ressource est valide, il organise son accès. Le bug apparaît quand la validation et l'utilisation ne sont pas dans la même fenêtre de synchronisation.

Chapter V

Aperçu du module

Ex	Titre	Difficulté	Points	
00	Modèle d'état du vérificateur	Moyen	4	
01	ALU32, bornes et variantes	Difficile	5	
02	Pruning et JIT	Difficile	5	
03	io_uring : PBUF et durée de vie	Difficile	5	
04	Patch gap et backport	Moyen	4	
	Total		**23**	

Chapter VI

Exercise 00

	Exercise 00
Modèle d'état du vérificateur	
Turn-in directory : `ex00/`	
Files to turn in : `verifier-state.md`	
Allowed functions : `grep, ctags, un éditeur de texte`	

Dans `verifier-state.md`, modélisez l'état d'un registre eBPF :

1. type (scalaire, pointeur, `OR_NULL`, etc.) ;
2. bornes minimales et maximales ;
3. précision 32/64 bits ;
4. provenance (stack, map, contexte, appel de helper) ;
5. contraintes de lecture et d'écriture.

Pour chaque champ, indiquez la fonction qui le met à jour et une passe qui peut l'invalider.



Le vérificateur ne suit pas une valeur réelle : il suit un ensemble d'états possibles. Une erreur de précision n'est pas une erreur de valeur, c'est une erreur sur l'ensemble.

Chapter VII

Exercise 01

	Exercise 01
ALU32, bornes et variantes	
Turn-in directory : `ex01/`	
Files to turn in : `alu32-variants.md`	
Allowed functions : `git, grep, un éditeur de texte`	

Étudiez la chaîne CVE-2020-27194 vers CVE-2021-3490. Le second correctif montre que le premier n'avait pas fermé la classe.

Mission

Dans `alu32-variants.md` :

1. expliquez la différence entre une opération 32 bits et une opération 64 bits sur le même registre ;

2. montrez comment les bornes 32 bits sont utilisées pour déduire une borne 64 bits ;

3. identifiez l'hypothèse qui manque dans le premier correctif ;

4. écrivez une requête `grep` qui trouve les fonctions de mise à jour des bornes ;

5. proposez un test de non-régression qui vérifie l'invariant, sans charger de programme dangereux.

Chapter VIII

Exercise 02

	Exercise 02
Pruning et JIT	
Turn-in directory : `ex02/`	
Files to turn in : `pruning-jit.md`	
Allowed functions : `git, grep, objdump, un éditeur de texte`	

Le pruning supprime des états jugés équivalents. CVE-2023-2163 montre qu'une équivalence locale peut être fausse globalement.

Mission

Dans `pruning-jit.md` :

1. décrivez la structure qui stocke les états visités ;
2. expliquez le critère d'équivalence utilisé par le pruning ;
3. donnez un exemple abstrait où deux états sont équivalents pour une passe mais pas pour une autre ;
4. comparez la sémantique du vérificateur et celle du JIT sur une opération arithmétique ;
5. listez trois endroits où le JIT peut introduire une différence (extension de signe, division, décalage).

Chapter IX

Exercise 03

	Exercise 03
io_uring : PBUF et durée de vie	
Turn-in directory : `ex03/`	
Files to turn in : `pbuf.md`	
Allowed functions : `grep, ctags, un éditeur de texte`	

Étudiez le cycle de vie d'un buffer enregistré et d'une requête io_uring. Les CVE-2023-2598 et CVE-2024-0582 touchent des ressources dont la durée de vie est mal protégée.

Mission

Dans `pbuf.md` :

1. décrivez les rings SQ/CQ et les buffers enregistrés ;
2. identifiez les points de validation et les points de libération ;
3. expliquez pourquoi la validation à la soumission ne suffit pas ;
4. proposez un modèle de propriété (« qui possède la ressource, quand la rend-elle ? ») ;
5. listez cinq questions d'audit pour un patch io_uring ;
6. expliquez pourquoi un test de course doit contrôler l'ordonnancement (communication points, injection de délai) plutôt que d'ajouter un `sleep()`.



L

es bugs de concurrence sont sensibles au timing. Une analyse statique peut donner une fausse impression de certitude. Indiquez toujours les fenêtres que vous n'avez pas prouvées. Le projet MAccConc de Project Zero montre qu'un test de course reproductible repose sur des accès mémoire observés et un ordre imposé, pas sur une attente arbitraire.

Chapter X

Exercise 04

	Exercise 04
Patch gap et backport	
Turn-in directory : `ex04/`	
Files to turn in : `backport.md`	
Allowed functions : `git, python3, un éditeur de texte`	

Choisissez trois CVE eBPF et deux CVE io_uring. Pour chacune, comparez le correctif amont et le backport dans une branche stable.

Mission

Dans `backport.md` :

1. listez les commits amont et stable ;
2. identifiez les différences de contexte ;
3. expliquez pourquoi un backport peut être incomplet ;
4. mesurez le délai entre amont et stable ;
5. comparez votre délai avec le patch gap OtterRoot (CVE-2024-26809) : commit public, publication de l'exploit KernelCTF, puis arrivée dans Ubuntu et Debian ;
6. concluez sur le risque d'exposition.

Chapter XI

Rendu et évaluation

```
A-EPI-003-ebpf-io-uring-audit/
|-- ex00/verifier-state.md
|-- ex01/alu32-variants.md
|-- ex02/pruning-jit.md
|-- ex03/pbuf.md
+-- ex04/backport.md
```

Critère	**Attendu**	**Points**
État	Modèle de registre complet	5
ALU32	Variante expliquée, test proposé	5
Pruning/JIT	Équivalence locale vs globale	5
io_uring	Propriété de durée de vie explicite	5
Backport	Comparaison amont/stable documentée	3

Chapter XII

Ressources par exercice

Exercice 00 — Modèle d'état du vérificateur

- Documentation eBPF : https://docs.kernel.org/bpf/verifier.html
- Papier, section 14.1.
- À chercher : `bpf_reg_state`, `tnum`, `regsafe`, `check_cfg`, `do_check`.

Exercice 01 — ALU32, bornes et variantes

- NVD : CVE-2020-27194, CVE-2021-3490, CVE-2022-23222.
- Papier, sections 14.2 à 14.4.
- Commits `5b9fbeb75b6a`, `049c4e13714e` et `64620e0a1e71`.
- À chercher : `scalar32_min_max_*`, `PTR_TO_ALLOC_MEM_OR_NULL`, `reg_type_may_be_null`.

Exercice 02 — Pruning et JIT

- NVD : https://nvd.nist.gov/vuln/detail/CVE-2023-2163
- Google VRP : https://bughunters.google.com/blog/6303226026131456/
- Weggli : https://github.com/weggli-rs/weggli
- Commit `71b547f56124`.
- À chercher : `!precise`, `backtrack_insn`, `JIT`, `bpf_jit_kallsyms`.

Exercice 03 — io_uring : PBUF et durée de vie

- NVD : CVE-2023-2598, CVE-2024-0582.
- Papier, section 15.3.
- io_uring : https://kernel.dk/io_uring.pdf
- À chercher : `io_sqe_buffer_register`, `IORING_REGISTER_PBUF_RING`, `IOU_PBUF_RING_MMAP`, `mmap`.

Exercice 04 — Patch gap et backport

- Project Zero, emergency patching : https://projectzero.google/2026/10/emergency-patching.html
- OtterSec, OtterRoot : https://osec.io/blog/netfilter-universal-root-1-day/
- NVD : https://nvd.nist.gov/vuln/detail/CVE-2024-26809
- À chercher : `upstream commit`, `stable`, `Ubuntu`, `Debian`, `activation`.

Chapter XIII

Sources

- Phrack, *Security is failure: Every Patch Is a Guess*, sections 6.1, 7 et 13.
- Google Project Zero, CVE-2023-2163.
- Project Zero, *Testing race conditions with memory access tracing and stack-based delay injection* : https://projectzero.google/2026/09/maccconc-race-condition.html
- OtterSec, *OtterRoot: Netfilter universal root 1-day* : https://osec.io/blog/netfilter-universal-root-1-day/
- Documentation du vérificateur eBPF et du JIT.
- NVD : CVE-2020-27194, CVE-2021-3490, CVE-2022-23222, CVE-2023-2598, CVE-2024-0582, CVE-2024-26809.